Relatório

Laboratórios de Informática III

Grupo 9:
Rafael Figueiras Esteves
João Pedro Cunha Cerqueira
Nuno Henrique Dias Pereira



Universidade do Minho
Escola de Engenharia

Introdução **3**
Sistema **4**
1. Dataset e Inputs 4
2.Interpretador/app_runner 4
3. Parsers 4
4.Gestores 4
Discussão **6**
1. Modularização e Encapsulamento 6
2. Escolha das Estruturas de Dados 6
Gestão de Entidades: Rápida e Segura 6
3. Queries e Código Simples 6
4. O Parser Genérico: Inteligência e Reutilização 7
5. Gravação de Erros: 7
6. Metodologia de Testes e Análise de Desempenho 7
Conclusão **8**

Introdução

Este trabalho prático de Laboratórios de Informática III tem como objetivo principal o desenvolvimento de competências essenciais na implementação estruturada de programas em linguagem C. O foco é a modularização e o encapsulamento, técnicas cruciais para a organização de projetos grandes, que serão exercitadas através da separação do código em ficheiros de interface (.h) e de implementação (.c).
O tema central do projeto é a criação de um sistema de gestão de aeroportos e voos, onde exploraremos um conjunto de dados extenso (voos, aeroportos, passageiros, reservas, etc.). O programa deverá ser capaz de carregar esta informação para estruturas de dados em memória e utilizá-la para responder a um conjunto de queries (interrogações).

Brevemente, destacamos que o nosso trabalho focará em:

- **Implementação Estruturada:** Garantir que o programa está dividido em módulos bem definidos (como sugerido na arquitetura de referência), assegurando o isolamento e o correto uso do encapsulamento.
- **Validação de Dados Abrangente:** Uma parte significativa do esforço inicial será dedicada à implementação robusta de validações sintáticas e lógicas para os dados de entrada (.csv), garantindo a integridade da informação processada
- **Mecanismos de Teste e Qualidade:** O desenvolvimento incluirá um programa de testes dedicado, que não só valida a correção das queries, mas também mede o tempo de execução e o uso de memória, elementos essenciais para garantir a qualidade do código.

Sistema

Na arquitetura do nosso programa foi adotado uma estrutura como a que está representada a figura.
A escolha desta estrutura foi sendo mudada ao longo do desenvolvimento do trabalho, chegando a esta forma final.

Organizamos o sistema em vários módulos distintos de modo a separar responsabilidades e facilitar a evolução e manutenção do código ao longo do projeto. A arquitetura baseia-se em quatro componentes principais: Inputs/Dataset, Interpretador, Parsers e Gestores, que interagem entre si de forma clara e consistente.

1. Dataset e Inputs

Começamos por agrupar todos os ficheiros de entrada: os CSV disponibilizados e o ficheiro input.txt que contém as queries. Esta separação permite centralizar tudo o que diz respeito a dados brutos num único ponto, facilitando o carregamento inicial.

2.Interpretador/app_runner

O app_runner liga os dados de entrada aos gestores e, depois, as queries ao ficheiro de comandos e aos resultados gerados.
A função carregar_dados trata de invocar os parsers corretos.
A função ler_inputs lê os comandos do ficheiro, chama executar_querry e encaminha o trabalho para o módulo de queries.
A separação do interpretador permitiu suportar dois modos de execução, o programa principal e o programa de testes, com uma lógica idêntica. Isto não só simplifica o sistema, como também garante a consistência entre os modos.

3. Parsers

No projeto temos um parser específico por entidade (aeronaves, aeroportos, voos, passageiros e reservas), que recebe uma linha CSV, valida-a e constrói a respetiva estrutura C. Cada módulo faz todo o ciclo de leitura do ficheiro: abre o CSV, usa getline, divide a linha com csv_split_quoted, regista eventuais erros num ficheiro dedicado e invoca o gestor apropriado para armazenar a entidade válida.

Na fig.1 a linha que liga os parsers ao gestores está em tracejado, pois para alguns casos é necessário passarmos o gestor global para resolver esses casos.

4.Gestores

Depois de passar pelo parser, as entidades são entregues aos gestores. Aqui optámos por usar coleções da GLib: GArray para preservar a ordem de leitura, GHashTable para permitir pesquisas rápidas por chave.

O módulo gestorglobal.c funciona como uma camada agregadora, fornecendo um ponto único de acesso a todos os gestores. Esta decisão surgiu da necessidade de na parte das validações termos de consultar alguns dados de outras entidades, por isso decidimos utilizar esta abordagem para tornar os dados consistentes e facilmente acessíveis a qualquer parte do programa, sem criar dependências cruzadas entre módulos.

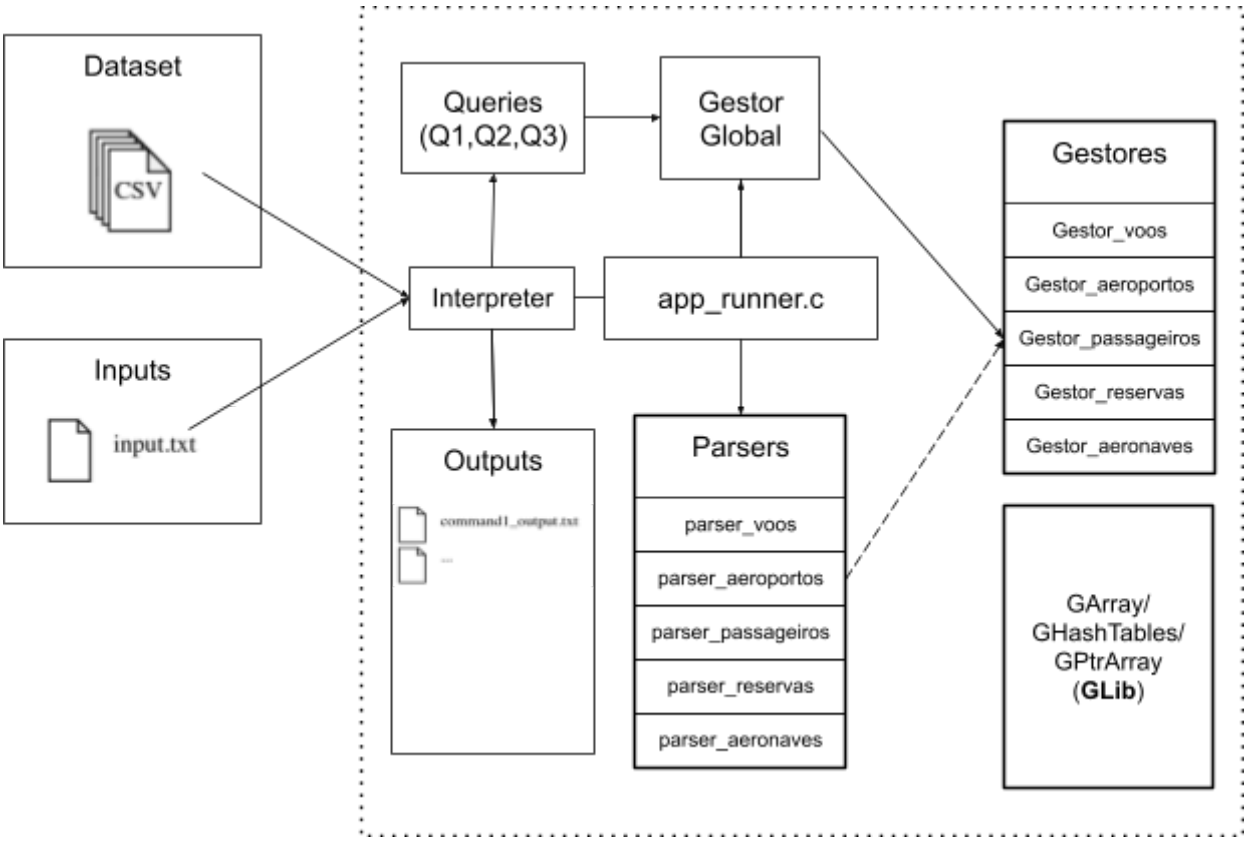


Figura 1: Esquema da arquitetura aplicada

Discussão

A primeira fase do trabalho exigiu não apenas a implementação das funcionalidades básicas do sistema, mas também a avaliação crítica das decisões de arquitetura e das estruturas usadas. Nesta seção analisamos essas escolhas, bem como o desempenho e a qualidade do sistema desenvolvido.

1. Modularização e Encapsulamento

Desde o início, procurámos organizar o projeto segundo princípios de modularização fortes. A divisão em módulos (Parsers, Gestores, Interpretador e Módulo de Queries) permitiu separar responsabilidades e tornar o código mais claro e facilmente extensível.

O encapsulamento foi assegurado através da utilização de ficheiros de interface (.h) que expõem apenas as funções estritamente necessárias, mantendo as estruturas internas e funções auxiliares ocultas nos ficheiros .c. Esta abordagem garantiu:

- Redução do acoplamento entre módulos.
- Maior segurança e robustez, evitando acessos indevidos aos dados internos.
- Capacidade de alterar internamente a implementação dos gestores ou parsers sem afetar outros componentes.

2. Escolha das Estruturas de Dados

A nossa escolha de estruturas de dados foi guiada por dois objetivos: máxima eficiência e clareza total na gestão de entidades. Assim, a escolha da GLib como ferramenta principal foi determinante.

Gestão de Entidades: Rápida e Segura

Em vez de usarmos arrays dinâmicos manuais, implementámos uma solução robusta:

- **GArray:** Usado para armazenar ponteiros para as nossas entidades, dando-nos redimensionamento automático e fiável.
- **GHashTable:** Garante acesso imediato a qualquer entidade através da sua chave (operações lookup).

Com esta implementação eliminamos o código repetitivo de gestão de memória e tornamos as operações de acesso muito mais seguras e rápidas.

3. Queries e Código Simples

A GLib é vantajosa na hora de processar dados. Na query 2, por exemplo, precisávamos de agregar voos e ordená-los.

- **Sem GLib:** Teríamos de implementar tabelas de dispersão e algoritmos de ordenação complexos do zero.
- **Com GLib:** Usámos uma GHashTable para acumular os dados e depois um GPtrArray com g_ptr_array_sort para ordenar os resultados instantaneamente.

Isto permitiu-nos focar 100% na lógica da query, em vez de nos perdermos em pormenores de implementação.

4. O Parser Genérico: Inteligência e Reutilização

Criámos um parser de CSV centralizado (csv_parser) que é uma verdadeira potência:

- **Faz o trabalho pesado:** Trata de todo o IO, divide campos de forma inteligente (incluindo tratamento de aspas com csv_split_quoted) e regista automaticamente linhas inválidas.
- **Design flexível:** O csv_parser não conhece os detalhes de cada entidade; limita-se a ler linhas e invocar os callbacks fornecidos para tratar da validação e armazenamento.

Uma das vantagens é zero duplicação de código. Todos os módulos beneficiam das mesmas regras de processamento e validação, garantindo consistência total.

5. Gravação de Erros

Usamos g_strndup no parser genérico. Isto copia a linha exata antes de qualquer normalização. Assim, o CSV de erros reflete exatamente o que foi lido, incluindo terminadores de linha. Esta abordagem resolve problemas de depuração de forma elegante, sem recorrer a buffers fixos que poderiam ser arriscados.

6. Metodologia de Testes e Análise de Desempenho

Características dos computadores dos integrantes do grupo

Aluno	**Processador**	**Memória Ram**
Rafael Esteves	AMD Ryzen™ 7 7840HS	32GB
João Cerqueira	Intel Core I7-13620H	16GB
Henrique Pereira	Intel Core Ultra 7 155H	32GB

Tempos de execução das Queries

Tempo	Rafael Esteves	João Cerqueira	Henrique Pereira
Q1	0.015 ms	0.018 ms	0.010 ms
Q2	90.173 ms	51.247 ms	34.219 ms
Q3	48.187 ms	33.431 ms	28.001 ms
Total:	5.735311 s	3.388250 s	2.489596 s

Memória utilizada (média)

Aluno	**Uso de Memória**
Rafael Esteves	562.07 MB
João Cerqueira	447.68 MB
Henrique Pereira	582.11 MB

Conclusão

O desenvolvimento desta primeira fase do trabalho prático permitiu cumprir integralmente o objetivo central de aplicação dos conceitos de modularização e encapsulamento. A arquitetura do nosso sistema foi cuidadosamente planeada, separando as funcionalidades em módulos coesos como Parser, Gestores de Entidades e Módulo de Queries, garantindo que o código está dividido de forma clara em ficheiros de interface (.h) e implementação (.c), e promovendo o isolamento entre eles. Esta estrutura bem definida é a fundação para o sucesso do projeto.

O esforço inicial foi concentrado em construir um programa robusto e fiável. Isto implicou a implementação de um sistema rigoroso de validação de dados que processa e filtra os ficheiros CSV, aplicando todas as regras sintáticas e lógicas exigidas, garantindo assim que apenas dados válidos são carregados em memória. Além disso, dedicamos especial atenção à qualidade do código e à gestão de recursos, confirmando a ausência de memory leaks (perdas de memória) através de testes com o Valgrind. O programa de testes é útil para validar a correção das queries e para medir o tempo de execução e o uso de memória.

Com a correta implementação das queries da Fase 1 e a consolidação da estrutura do programa, a nossa principal lição aprendida é que a fase de planeamento e design modular é inseparável da qualidade do código final. Em conclusão, a Fase 1 forneceu a base estrutural e a prova de conceito necessárias para avançar com confiança para os desafios de otimização e complexidade da Fase 2.